

Lecture 15: Gaussian Elimination

Instructors:

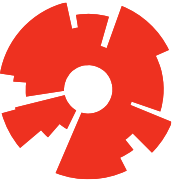
Christoph Lohrmann (christophl@ru.is)

[Based on slides for „Decoding the Matrix“]



WEEK





Agenda

Chapter 7

Lecture 15:

- Gaussian Elimination - Origins & Uses
- Echelon Form of a Matrix
- Uses of Echelon Form
- Transforming a Matrix to Echelon Form
- Elementary Row Addition Operations
- Gaussian Elimination
- Gaussian Elimination in $GF(2)$
- LU Factorization
- Preserving the Row Space





Recap: Some Key Take-aways (Week 7)

There can be different bases for the same vector space $\mathcal{V}$ but all have the same size

✓ If you have a basis,
there is no smaller
other basis

„**Dimension**“ of a vector space is the size of a basis for that vector space. „**Rank**“ of a set of vectors is the dimension of its span. For a set of D -vectors, the rank is at most $|D|$

How many linearly
independent vectors
needed to span space

Every (finite) set of vectors has a subset that is a basis for the span of these vectors.

[Subset-Basis Lemma]

We can find a basis
for the span

For any n -column matrix A , $n = \text{rank } A + \text{nullity } A$ where *nullity* A refers to the dimension of the Null Space of A *[Rank-Nullity Theorem]*

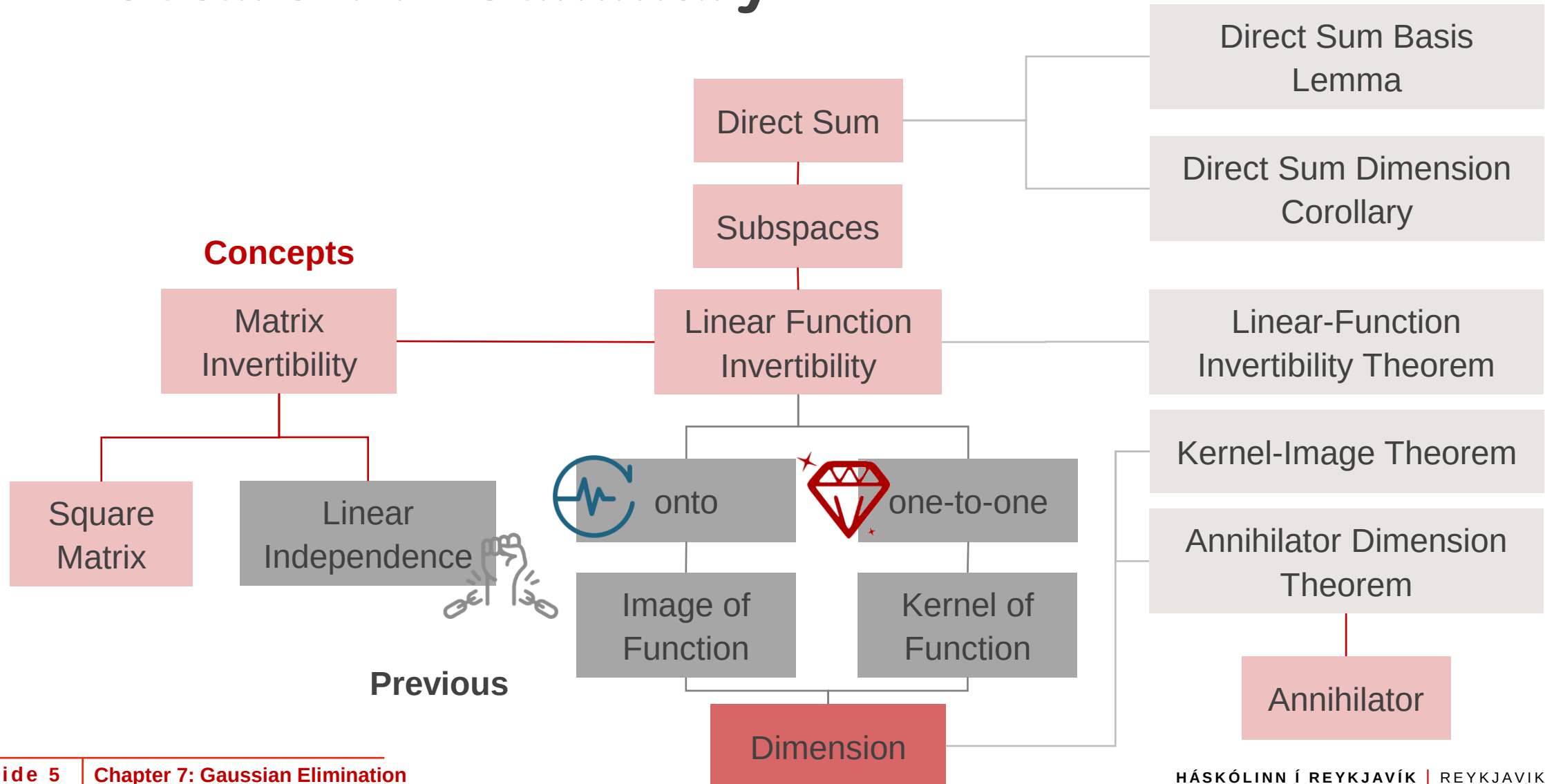
} Linear independence
→ $n = \text{rank } A$

A **matrix is invertible** iff it is a square matrix, and its columns are linearly independent

} Very Practical!



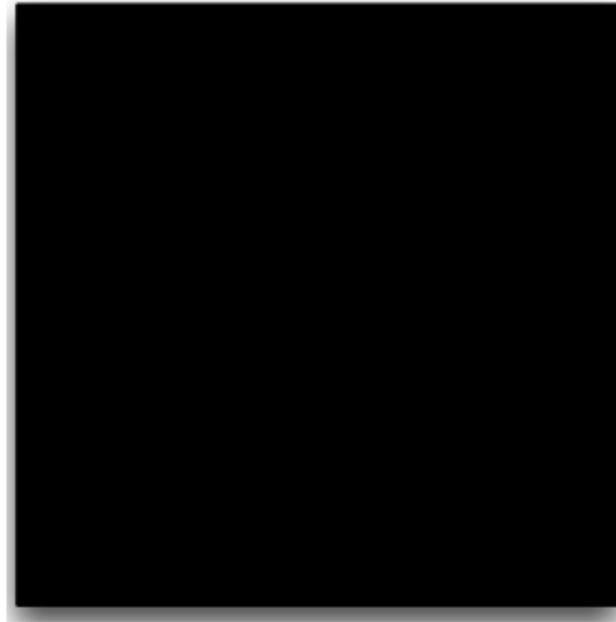
Lecture 14 - Summary





Solve - The Blackbox

So far `solve(A, b)` is a black box.



With Gaussian elimination, we begin to find out what's inside.





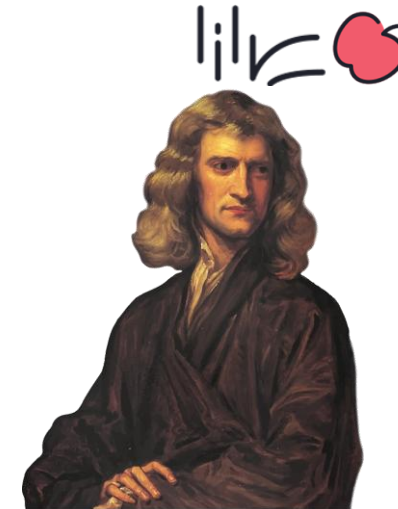
Gaussian Elimination - Origins

Method illustrated in Chapter Eight of a Chinese text, *The Nine Chapters on the Mathematical Art*, that was written roughly two thousand years ago.

Rediscovered in Europe by Isaac Newton (England) and Michel Rolle (France)

Gauss called the method *eliminatio nem vulgarem* ("common elimination")

Gauss adapted the method for another problem (one we study soon) and developed notation.



Improvise, adapt, ...
eliminate

Latin *vulgaris*
= "common" or "of
the common people"





Gaussian Elimination - Uses

- ▶ *Finding a basis for the span of given vectors.* This additionally gives us an algorithm for rank and therefore for testing linear dependence.
- ▶ *Solving a matrix equation,* which is the same as *expressing a given vector as a linear combination of other given vectors*, which is the same as *solving a system of linear equations*
- ▶ *Finding a basis for the null space of a matrix,* which is the same as *finding a basis for the solution set of a homogeneous linear system*, which is also relevant to representing the solution set of a general linear system.
- ▶ Finding the *inverse matrix for a matrix* using an augmented matrix **[Gauss-Jordan Method]**



Echelon Form of a Matrix (I)

Echelon form a generalization of triangular matrices

Note that

- ▶ the first nonzero entry in row 0 is in column 1,
- ▶ the first nonzero entry in row 1 is in column 2,
- ▶ the first nonzero entry in row 2 is in column 4, and
- ▶ the first nonzero entry in row 4 is in column 5.

Example:

$$\begin{array}{c}
 c_0 \quad c_1 \quad c_2 \quad c_3 \quad c_4 \quad c_5 \\
 \left[\begin{array}{cccccc}
 0 & 2 & 3 & 0 & 5 & 6 \\
 0 & 0 & 1 & 0 & 3 & 4 \\
 0 & 0 & 0 & 0 & 1 & 2 \\
 0 & 0 & 0 & 0 & 0 & 9
 \end{array} \right]
 \end{array}$$

Echelon form but
not triangular matrix

Definition: An $m \times n$ matrix A is in *echelon form* if it satisfies the following condition: for any row, if that row's first nonzero entry is in position k then every previous row's first nonzero entry is in some position less than k .



Echelon Form of a Matrix (II)

Definition: An $m \times n$ matrix A is in *echelon form* if it satisfies the following condition: for any row, if that row's first nonzero entry is in position k then every previous row's first nonzero entry is in some position less than k .

This definition implies that, as you iterate through the rows of A , the first nonzero entries per row move strictly right, **forming a sort of staircase** that descends to the right.

2	1	0	4	1	3	9	7
0	6	0	1	3	0	4	1
0	0	0	0	2	1	3	2
0	0	0	0	0	0	0	1

2	1	0	4	1	3	9	7
>	6	0	1	3	0	4	1
>	>	0	0	2	1	3	2
>	>	0	0	>	0	0	1



Echelon Form of a Matrix (III)

Definition: An $m \times n$ matrix A is in *echelon form* if it satisfies the following condition: for any row, if that row's first nonzero entry is in position k then every previous row's first nonzero entry is in some position less than k .

$$\begin{bmatrix} 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 0 & 9 \end{bmatrix}$$

Echelon Form
(not triangular)

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 3 & 0 & 1 \end{bmatrix}$$

Not Echelon Form
(but a re-ordering can fix that)

$$\begin{bmatrix} 0 & 4 & 1 & 3 & 0 & 6 \\ 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 0 & 0 & 0 & 9 \end{bmatrix}$$

Not Echelon Form
(re-ordering won't fix that)

$$\begin{bmatrix} 4 & 1 & 3 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 9 \end{bmatrix}$$

Echelon Form
(even triangular)



Use of Echelon Form (I)

If a row of a matrix in echelon form is all zero then every subsequent row must also be all zero, e.g.

$$\begin{bmatrix} 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$



I tolerate no non-zero entries below me

What good is it having a matrix in echelon form?

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.





Use of Echelon Form (II)

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.

For example, a basis for the row space of

$$\begin{bmatrix} 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

is $\{[0, 2, 3, 0, 5, 6], [0, 1, 0, 3, 4]\}$.

B1: Spanning

B2: Linearly Independent



Use of Echelon Form (III)

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.

$$\begin{bmatrix} 0 & 2 & 3 & 0 & 5 & 6 \\ 0 & 0 & 1 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 0 & 0 & 9 \end{bmatrix}$$

All rows in this echelon form matrix contain **nonzero** entries
so all are needed to form a basis for the row space

The basis for the row space is $\{[0, 2, 3, 0, 5, 6], [0, 0, 1, 0, 3, 4], [0, 0, 0, 0, 1, 2], [0, 0, 0, 0, 0, 9]\}$



Use of Echelon Form (IV)

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.

B1

It is obvious that the nonzero rows span the row space. We need only show that these

B2

vectors are linearly independent. We prove it using the Grow algorithm:

```
def GROW( $\mathcal{V}$ )
   $S = \emptyset$ 
  repeat while possible:
    find a vector  $\mathbf{v}$  in  $\mathcal{V}$  that is not in  $\text{Span } S$ , and put it in  $S$ 
```



$$\begin{bmatrix} 4 & 1 & 3 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 9 \end{bmatrix} \uparrow$$

We run the Grow algorithm, adding rows of matrix in reverse order to S

(as it is more obvious this way that they are
linearly independent)



Use of Echelon Form (V)

$$\begin{bmatrix} 4 & 1 & 3 & 0 \\ 0 & 3 & 0 & 1 \\ 0 & 0 & 1 & 7 \\ 0 & 0 & 0 & 9 \end{bmatrix}$$

We run the Grow algorithm, adding rows of matrix in reverse order to S :

- ▶ Since $\text{Span } \emptyset$ does not include $[0, 0, 0, 9]$, the algorithm adds this vector to S .
- ▶ Now $S = \{[0, 0, 0, 9]\}$. Every vector in $\text{Span } S$ has zeroes in positions 0, 1, 2, so $\text{Span } S$ does not contain $[0, 0, 1, 7]$, so the algorithm adds this vector to S .
- ▶ Now $S = \{[0, 0, 0, 9], [0, 0, 1, 7]\}$. Every vector in $\text{Span } S$ has zeroes in positions 0, 1, so $\text{Span } S$ does not contain $[0, 3, 0, 1]$, so the algorithm adds it.
- ▶ Now $S = \{[0, 0, 0, 9], [0, 0, 1, 7], [0, 3, 0, 1]\}$. Every vector in $\text{Span } S$ has a zero in position 0, so $\text{Span } S$ does not contain $[4, 1, 3, 0]$, so the algorithm adds it, and we are done.



Transforming a Matrix to Echelon Form

Suggests an approach: To find basis for row space of a matrix A , iteratively transform A into a matrix B

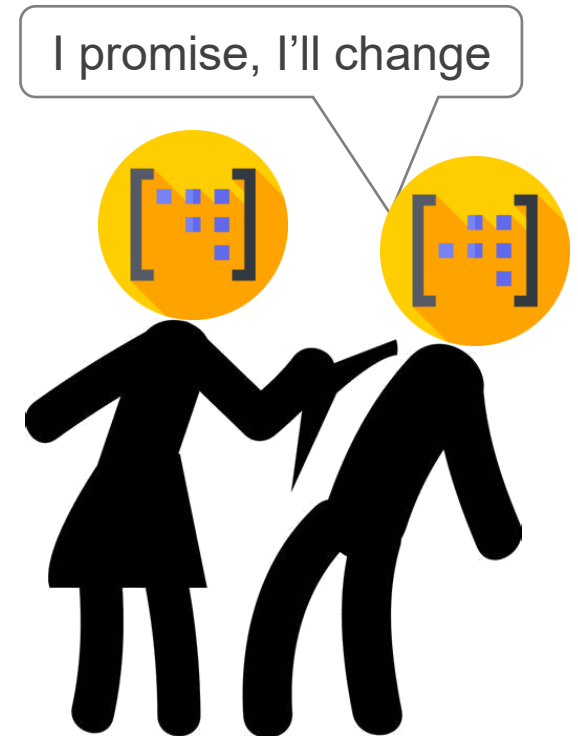
- ▶ in echelon form
- ▶ with no zero rows
- ▶ whose row space is the same as that of A .

We will represent current matrix as a rowlist.

Assume `rowlist` has been initialized with a list of Vecs, e.g..

`rowlist` = $\left[\begin{array}{ccc} A & B & C \\ 0 & 1 & 2 \end{array}, \begin{array}{ccc} A & B & C \\ 1 & 2 & 3 \end{array}, \begin{array}{ccc} A & B & C \\ 0 & 0 & 1 \end{array} \right]$

We will mutate this variable.





First Attempt - Sort Rows (I)

Goal: Transform a matrix `rowlist` into a matrix `new_rowlist` in echelon form.

Here's an easy matrix to start with:

$$A$$

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	0	0	0	1	2
1	0	2	3	0	5	6
2	0	0	0	0	0	0
3	0	0	1	0	3	4

Step 1

Step 2

$$B$$

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	2	3	0	5	6

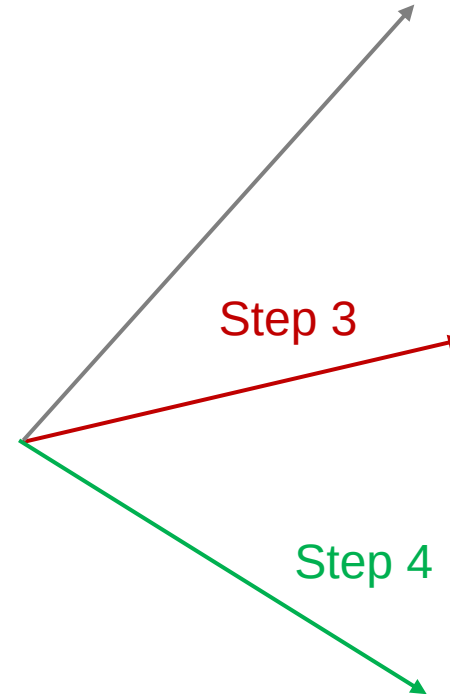
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	2	3	0	5	6
1	0	0	1	0	3	4

⋮



First Attempt - Sort Rows (II)

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	0	0	0	1	2
1	0	2	3	0	5	6
2	0	0	0	0	0	0
3	0	0	1	0	3	4



⋮

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	2	3	0	5	6
1	0	0	1	0	3	4
2	0	0	0	0	1	2

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
0	0	2	3	0	5	6
1	0	0	1	0	3	4
2	0	0	0	0	1	2
3	0	0	0	0	0	0

Suggests an algorithm: sort the rows according to position of leftmost nonzero entry.



First Attempt - Sort Rows (III)

Goal: a method of transforming a rowlist into one that is in echelon form.

First attempt: Sort the rows by position of the leftmost nonzero entry.

We will use a naive algorithm of sorting:

- ▶ first choose a row with a nonzero in first column,
- ▶ then choose a row with a nonzero in second column,
- ⋮

accumulating these in a list `new_rowlist`, initially empty: `new_rowlist = []`



First Attempt - Sort Rows (IV)

```
col_label_list = sorted(rowlist[0].D, key=str)
new_rowlist = []
rows_left = set(range(len(rowlist)))
```

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    pivot = rows_with_nonzero[0]
    new_rowlist.append(rowlist[pivot])
    rows_left.remove(pivot)
```

- ▶ Algorithm iterates through the column labels in order.
- ▶ In each iteration, algorithm finds a list `rows_with_nonzero` of indices of the remaining rows that have nonzero entries in the current column
- ▶ Algorithm selects one of these rows (the *pivot row*), adds it to `new_rowlist`, and removes its index from `rows_left`.



First Attempt - Sort Rows (V)

```
col_label_list = sorted(rowlist[0].D, key=str)
new_rowlist = []
rows_left = set(range(len(rowlist)))
```

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    pivot = rows_with_nonzero[0]
    new_rowlist.append(rowlist[pivot])
    rows_left.remove(pivot)
```

Let's try this out!

Run the algorithm on

$$\begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 0 & 5 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 4 & 5 \end{bmatrix}$$



First Attempt - Sort Rows (VI)

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    pivot = rows_with_nonzero[0]
    new_rowlist.append(rowlist[pivot])
    rows_left.remove(pivot)
```

(Just the
main loop
here)

0	2	3	4	5
0	0	0	0	5
1	2	3	4	5
0	0	0	4	5

→ Iteration 1 - First Column (0)

rows_with_nonzero ✓
new_rowlist is $[1, 2, 3, 4, 5]$
rows_left is $\{0, 1, 3\}$

→ Iteration 2 – Second Column (1)

rows_with_nonzero ✓
new_rowlist is $[1, 2, 3, 4, 5], [0, 2, 3, 4, 5]$
rows_left is $\{1, 3\}$

Iteration 3 - Third Column (2)

rows_with_nonzero ✗

Problem: No rows with a nonzero value in the third column (2)

→ Algorithm should move on to next column without changing

new_rowlist rows_left



First Attempt - Sort Rows (VII)

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    pivot = rows_with_nonzero[0]
    new_rowlist.append(rowlist[pivot])
    rows_left.remove(pivot)
```

(Just the
main loop
here)

New Version

Move to next iteration in case there is no row with nonzero value in the current column (c)

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    → if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        new_rowlist.append(rowlist[pivot])
        rows_left.remove(pivot)
```



First Attempt - Sort Rows (VIII)

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        new_rowlist.append(rowlist[pivot])
        rows_left.remove(pivot)
```

(New Version -
Just the main
loop here)

0	2	3	4	5
0	0	0	0	5
1	2	3	4	5
0	0	0	4	5

→ Iteration 3 - Third Column (2) [Skip – no changes made]

→ Iteration 4 - Fourth Column (3)

→ rows_with_nonzero ✓

new_rowlist is $[[1, 2, 3, 4, 5], [0, 2, 3, 4, 5], [0, 0, 0, 4, 5]]$

rows_left is {1}

Iteration 5 - Fourth Column (4)

rows_with_nonzero ✓

new_rowlist is $[[1, 2, 3, 4, 5], [0, 2, 3, 4, 5], [0, 0, 0, 4, 5], [0, 0, 0, 0, 5]]$

rows_left is $\{\emptyset\}$



First Attempt - Sort Rows - Issue

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        new_rowlist.append(rowlist[pivot])
        rows_left.remove(pivot)
```

New Example

rowlist					
0	2	3	4	5	
0	0	0	3	2	
1	2	3	4	5	
0	0	0	6	7	

 $\Rightarrow$

new_rowlist					
1	2	3	4	5	
0	2	3	4	5	
0	0	0	3	2	
0	0	0	6	7	

Result is not in echelon form.





Gaussian Elimination (I)

Applying elementary row-addition operations does not change the row space. 😊

(Otherwise, the rows of the echelon form would not be a basis for the span of the original matrix)

Incorporate into the algorithm

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        rows_left.remove(pivot)
        new_rowlist.append(rowlist[pivot])
        add suitable multiple of rowlist[pivot] to each row in rows_with_nonzero[1:]
```

More Details please!





Gaussian Elimination (II)

Applying elementary row-addition operations does not change the row space. 😊

Incorporate into the algorithm

```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        rows_left.remove(pivot)
        new_rowlist.append(rowlist[pivot])
        for r in rows_with_nonzero[1:]:
            multiplier = rowlist[r][c]/rowlist[pivot][c]
            rowlist[r] -= multiplier * rowlist[pivot]
```

(determine multiple)
(subtract row multiple)

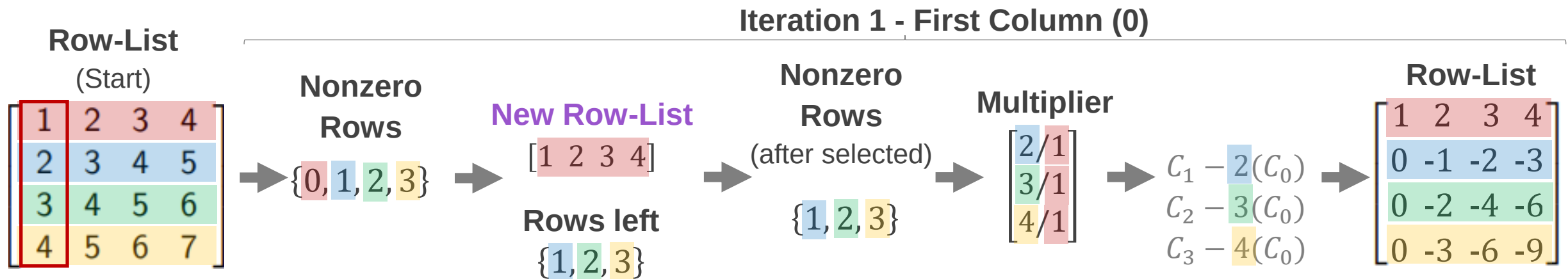
...about time! Thank you!





Gaussian Elimination - Example (I)

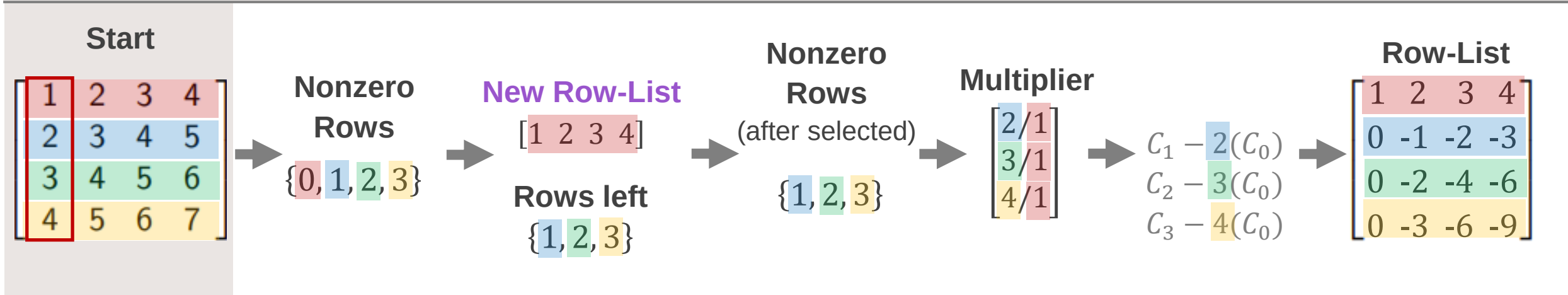
```
for c in col_label_list:
    rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
    if rows_with_nonzero != []:
        pivot = rows_with_nonzero[0]
        rows_left.remove(pivot)
        new_rowlist.append(rowlist[pivot])
        for r in rows_with_nonzero[1:]:
            multiplier = rowlist[r][c]/rowlist[pivot][c]
            rowlist[r] -= multiplier * rowlist[pivot]
```



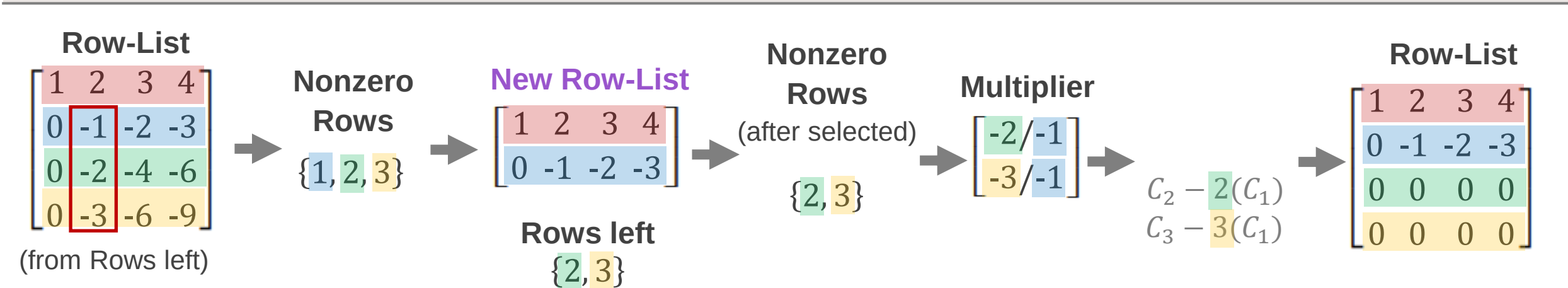


Gaussian Elimination - Example (II)

Iteration 1 - First Column (0)



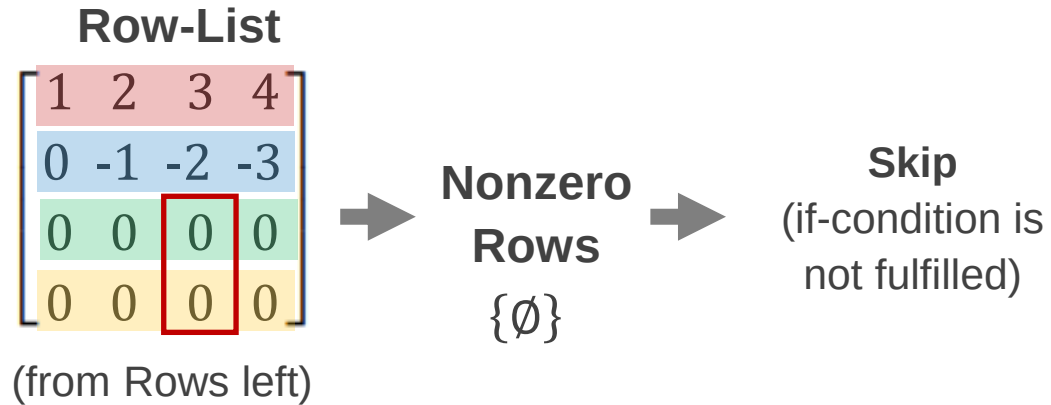
Iteration 2 - Second Column (1)



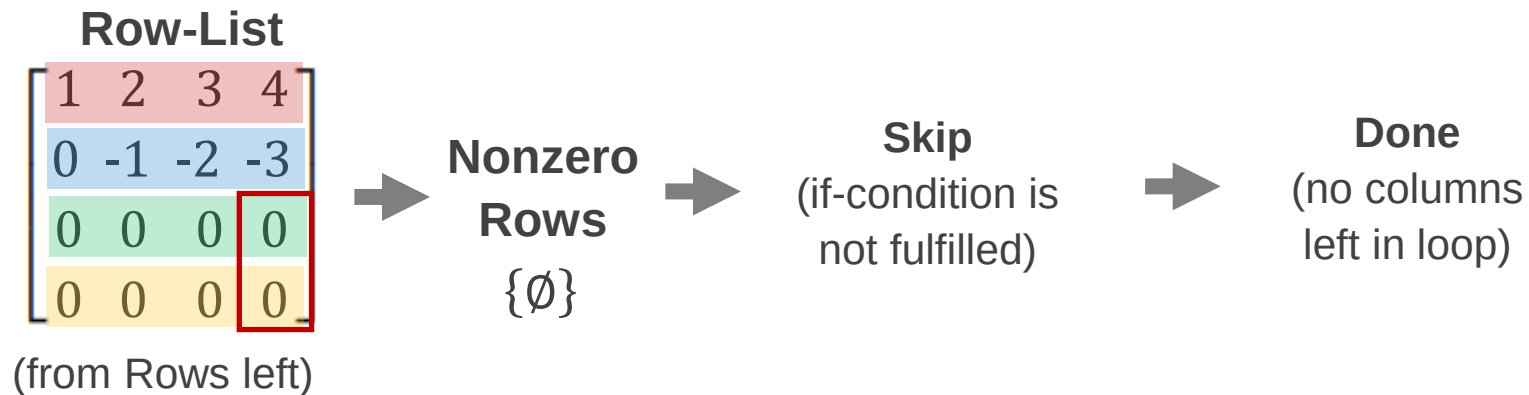


Gaussian Elimination - Example (III)

Iteration 3 - Third Column (2)



Iteration 4 - Fourth Column (3)





Gaussian Elimination - Example (IV)

Summary (End of each Iteration)

	(No Change)		(No Change)	
Start	Iteration 1	Iteration 2	Iteration 3	Iteration 4
Row-List [1 2 3 4] [2 3 4 5] [3 4 5 6] [4 5 6 7]	Row-List [1 2 3 4] [0 -1 -2 -3] [0 -2 -4 -6] [0 -3 -6 -9] New Row-List [1 2 3 4]	Row-List [1 2 3 4] [0 -1 -2 -3] [0 0 0 0] [0 0 0 0] New Row-List [1 2 3 4] [0 -1 -2 -3]	Row-List [1 2 3 4] [0 -1 -2 -3] [0 0 0 0] [0 0 0 0] New Row-List [1 2 3 4] [0 -1 -2 -3]	Row-List [1 2 3 4] [0 -1 -2 -3] [0 0 0 0] [0 0 0 0] New Row-List [1 2 3 4] [0 -1 -2 -3]

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.



Gaussian Elimination in GF(2) (I)

	A	B	C	D
0	0	0	1	1
✓ 1	1	0	1	1
2	1	0	0	1
3	1	1	1	1

A: Select row 1 as pivot.
Put it in `new_rowlist`
Since rows 2 and 3 have nonzeros, we must add row 1 to rows 2 and 3.

New Row-List

1	0	1	1
---	---	---	---

	A	B	C	D
0	0	0	1	1
✓ 1	1	0	1	1
2	0	0	1	0
✓ 3	0	1	0	0

B: Select row 3 as pivot.
Put it in `new_rowlist`
Other remaining rows have zeroes in column B, so no row additions needed.

New Row-List

1	0	1	1
0	1	0	0



Gaussian Elimination in GF(2) (II)

		A	B	C	D
✓	0	0	0	1	1
✓	1	1	0	1	1
	2	0	0	1	0
✓	3	0	1	0	0

C: Select **row 0** as pivot .
Put it in `new_rowlist`.
Only other remaining row
is **row 2**, and we add
row 0 to row 2.

New Row-List

1	0	1	1
0	1	0	0
0	0	1	1

		A	B	C	D
✓	0	0	0	1	1
✓	1	1	0	1	1
✓	2	0	0	0	1
✓	3	0	1	0	0

D: Only remaining row is
row 2, so select it as pivot
row.
Put it in `new_rowlist`
No other rows, so no row
additions.

New Row-List

1	0	1	1
0	1	0	0
0	0	1	1
0	0	0	1

[END]



Elementary Row-Addition Operations (I)

Need to introduce another transformation.... Repair the problem by changing the rows:

$$\begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & \textcircled{3} & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 6 & 7 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 3 & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & \textcircled{0} & 3 \end{bmatrix}$$

Subtract twice the second row $2[0, 0, 0, 3, 2]$ from the fourth getting new fourth row

$$[0, 0, 0, 6, 7] - 2[0, 0, 0, 3, 2] = [0, 0, 0, 6 - 6, 7 - 4] = [0, 0, 0, 0, 3]$$

The $\textcircled{3}$ in the second row is called the *pivot element*.
That element is used to zero out another element
in same column.

We know row-addition can be
done by matrix multiplication ...





Elementary Row-Addition Operations (II)

$$\begin{array}{c}
 \begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 3 & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 6 & 7 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 3 & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 0 & 3 \end{bmatrix} \\
 A \qquad \qquad \qquad U
 \end{array}$$

Transformation is multiplication by a *elementary row-addition matrix*:

Subtract twice the second row from the fourth one

$$\begin{array}{c}
 \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 3 & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 6 & 7 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 3 & 2 \\ 1 & 2 & 3 & 4 & 5 \\ 0 & 0 & 0 & 0 & 3 \end{bmatrix} \\
 M \qquad \qquad \qquad A \qquad \qquad \qquad U
 \end{array}$$



Elementary Row-Addition Operations (III)

How about changing the order of columns?

A $\begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix}$	Identity A A $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix}$
A U $\begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \xRightarrow{\text{(Change order)}} \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix}$	M_1 A U_1 $\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix}$
U_1 U_2 $\begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} \xRightarrow{R_2 + R_1} \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix}$	M_2 U_1 U_2 $\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix}$



Elementary Row-Addition Operations (IV)

$$\begin{array}{c} U_1 \\ \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} \end{array} \xRightarrow{R_2 + R_1} \begin{array}{c} U_2 \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix} \end{array}$$

$$\begin{array}{c} M_2 \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{array} \times \begin{array}{c} U_1 \\ \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} \end{array} = \begin{array}{c} U_2 \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix} \end{array}$$

$$\begin{array}{c} U_2 \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix} \end{array} \xRightarrow{R_3 - 0.5R_2} \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

$$\begin{array}{c} M_3 \\ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix} \end{array} \times \begin{array}{c} U_2 \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 3 & 3 \end{bmatrix} \end{array} = \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

Echelon Form

Nice! We used multiple matrices to get from the Original A to our final matrix B . Can we represent this differently?



Elementary Row-Addition Operations (V)

$$\begin{array}{c} A \\ \left[\begin{array}{ccc} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{array} \right] \end{array} \Rightarrow \begin{array}{c} U \\ \left[\begin{array}{ccc} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{array} \right] \end{array}$$

Welcome back old friend,
functional composition

$$\begin{array}{c} M_3 \\ \left[\begin{array}{ccc} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{array} \right] \end{array} \times \left(\begin{array}{c} M_2 \\ \left[\begin{array}{ccc} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{array} \right] \end{array} \times \left(\begin{array}{c} M_1 \\ \left[\begin{array}{ccc} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{array} \right] \end{array} \times \begin{array}{c} A \\ \left[\begin{array}{ccc} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{array} \right] \end{array} \right) \right) = \begin{array}{c} U \\ \left[\begin{array}{ccc} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{array} \right] \end{array}$$

First Step

Second Step

Third Step





Elementary Row-Addition Operations (V)

$$\begin{array}{c}
 \begin{array}{ccc} M_3 & & \\ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix} & \times & \left(\begin{array}{ccc} M_2 & & \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \times & \left(\begin{array}{ccc} M_1 & & \\ \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} & \times & \begin{array}{c} A \\ \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \end{array} \right) \end{array} \right) \\
 \text{First Step} \\
 \text{Second Step} \\
 \text{Third Step}
 \end{array}
 \end{array}
 = \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

$$\begin{array}{c}
 \begin{array}{ccc} M_3 & & \\ \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix} & \times & \begin{array}{ccc} M_2 & & \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} & \times & \begin{array}{ccc} M_1 & & \\ \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{array} \end{array} \end{array}
 \begin{array}{c} A \\ \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \end{array}
 = \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

**Associativity of
Matrix
Multiplication**



Elementary Row-Addition Operations (VI)

$$\begin{array}{c} M_3 \\ \left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix} \right. \\ \\ \\ \end{array} \times \begin{array}{c} M_2 \\ \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \end{array} \times \begin{array}{c} M_1 \\ \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \end{array} \left. \right) \begin{array}{c} A \\ \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \end{array} = \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

$$\begin{array}{c} M \\ \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & -0.5 & -0.5 \end{bmatrix} \end{array} \times \begin{array}{c} A \\ \begin{bmatrix} 0 & 3 & 3 \\ -2 & 5 & 5 \\ 2 & 1 & 3 \end{bmatrix} \end{array} = \begin{array}{c} U \\ \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix} \end{array}$$

One matrix to
do it all



Excursus: LU Factorization (I)

(Same example but without need to switch the rows)

$$\left(\begin{matrix} M_3 \\ M_2 \end{matrix} \right) \times \begin{matrix} A \\ U \end{matrix} = \begin{matrix} U \end{matrix}$$

$$\left(\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.5 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix}$$

$$M_3 M_2 A = U$$

$$MA = U$$

$$A = M^{-1}U = LU$$

$$\begin{matrix} A \\ U \end{matrix} = \begin{matrix} M^{-1} = L \end{matrix} \times \begin{matrix} U \end{matrix}$$

$$\begin{bmatrix} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0.5 & 1 \end{bmatrix} \times \begin{bmatrix} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{bmatrix}$$



Excursus: LU Factorization (II)

Any representation of a matrix as a product of two or more other matrices is called a **matrix factorization**

$$A = LU$$

$$\begin{array}{c} A \\ \left[\begin{array}{ccc} 2 & 1 & 3 \\ -2 & 5 & 5 \\ 0 & 3 & 3 \end{array} \right] \end{array} = \begin{array}{c} L \\ \left[\begin{array}{ccc} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0.5 & 1 \end{array} \right] \end{array} \times \begin{array}{c} U \\ \left[\begin{array}{ccc} 2 & 1 & 3 \\ 0 & 6 & 8 \\ 0 & 0 & -1 \end{array} \right] \end{array}$$

Lower
Triangular
Matrix

Upper
Triangular
Matrix



Preserving the Row Space (I)

We will show:

Proposition: If $MA = B$ where M is invertible then $\text{Row } A = \text{Row } B$.

Therefore change to row causes no change in row space.

Therefore basis for changed rowlist is also a basis for original rowlist.

Lemma: $\text{Row } NA \subseteq \text{Row } A$.



Preserving the Row Space (II)

Lemma: $\text{Row } NA \subseteq \text{Row } A.$



Proof: Let $\mathbf{v}$ be any vector in $\text{Row } NA$.

That is, $\mathbf{v}$ is a linear combination of the rows of NA .

By the linear-combinations definition of vector-matrix multiplication, there is a vector $\mathbf{u}$ such that

The product is a matrix

Thus, this is a linear combination

by associativity

$$\begin{aligned} \text{Imagine } u^T &= [\alpha_1, \alpha_2, \dots] \\ \mathbf{v} &= \begin{bmatrix} \mathbf{u}^T \end{bmatrix} \left(\begin{bmatrix} N \end{bmatrix} \begin{bmatrix} A \end{bmatrix} \right) \\ \text{Imagine } u^T N &= [\beta_1, \beta_2, \dots] \\ &= \left(\begin{bmatrix} \mathbf{u}^T \end{bmatrix} \begin{bmatrix} N \end{bmatrix} \right) \begin{bmatrix} A \end{bmatrix} \end{aligned}$$

which shows that $\mathbf{v}$ can be written as a linear combination of the rows of A .

QED



Preserving the Row Space (II)

Lemma: $\text{Row } NA \subseteq \text{Row } A$.

Proposition: If M is invertible then $\text{Row } MA = \text{Row } A$



Proof: Must show $\text{Row } MA \subseteq \text{Row } A$ and $\text{Row } A \subseteq \text{Row } MA$

► Lemma shows $\text{Row } MA \subseteq \text{Row } A$.

► Let $B = MA$

► M has an inverse $M^{-1} \Rightarrow M^{-1}B = A$

Since $M^{-1}MA = IA$

► Lemma shows $\text{Row } \underbrace{M^{-1}B}_A \subseteq \text{Row } \underbrace{B}_{MA}$

Obviously, M^{-1} is invertible

► That is, $\text{Row } A \subseteq \text{Row } MA$

QED



Exercise: Gaussian Elimination

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix}$$

You are given the following matrix A :

- 1) What is the matrix M_1 that reorders the rows (order: 3, 2, 1, 4)? What is the resulting matrix $M_1A = B_1$?
- 2) What is the matrix M_2 that performs the elementary row additions to ensure that for B_1 there is only one row that starts with a non-zero entry in the second column? Determine $M_2B_1 = B_2$.
- 3) What is the matrix M_3 that ensures that for B_2 there is only one row that starts with a non-zero entry in the third column? Determine $M_3B_2 = B$.
- 4) What is the matrix that performs all the steps in (1), (2) and (3) to get from A to B ? Show work.
- 5) What are the basis vectors for the row space of A ? Please state them explicitly one by one. Explain how you knew they were the basis vectors.



Solution

Exercise: Gaussian Elimination

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix}$$

a)

$$M_1 A = \begin{bmatrix} 0 & 0 & \textcircled{1} & 0 \\ 0 & 1 & 0 & 0 \\ \textcircled{1} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 5 \end{bmatrix} = B_1$$

b)

$$M_2 B_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & \textcircled{-2} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 5 \end{bmatrix} = B_2$$

$$R_3 - 2 R_2$$

c)

$$M_3 B_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \textcircled{-5} & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = B$$

$$R_4 - 5 R_3$$



Solution

Exercise: Gaussian Elimination

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix}$$

d)

$$M_3 M_2 M_1 A = \left(\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -5 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \right) \times \begin{bmatrix} 0 & 2 & 1 \\ 0 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} = B$$

$$M_3 M_2 M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -5 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -2 & 1 & 0 \\ 0 & 10 & -5 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & -2 & 0 & 0 \\ -5 & 10 & 0 & 1 \end{bmatrix}$$

e) If a matrix is in echelon form, the nonzero rows form a basis for the row space.

Thus, the basis vectors for the row space of A are the three vectors $[1, 3, 1]$, $[0, 1, 0]$, $[0, 0, 1]$



Some Key Take-aways (Lecture 15)

Gaussian Elimination, termed “common elimination” by Karl Friedrich Gauss, can be used to find a basis for a span of vectors or for the null space of a matrix and for solving a matrix equation

Gaussian Elimination brings a matrix into **Echelon form**, which means that if for a row the first nonzero entry is in position k , then the previous row's first nonzero entry needs to be in some position less than k (at least $k-1$).

Special Case:
Triangular Matrix

The nonzero rows in an Echelon form matrix are a **basis for the row space**

Sorting rows by itself may often not be enough to bring a matrix in Echelon form. Additional steps include using **scalar vector multiplication** and **vector addition** to add multiples of a row to another row

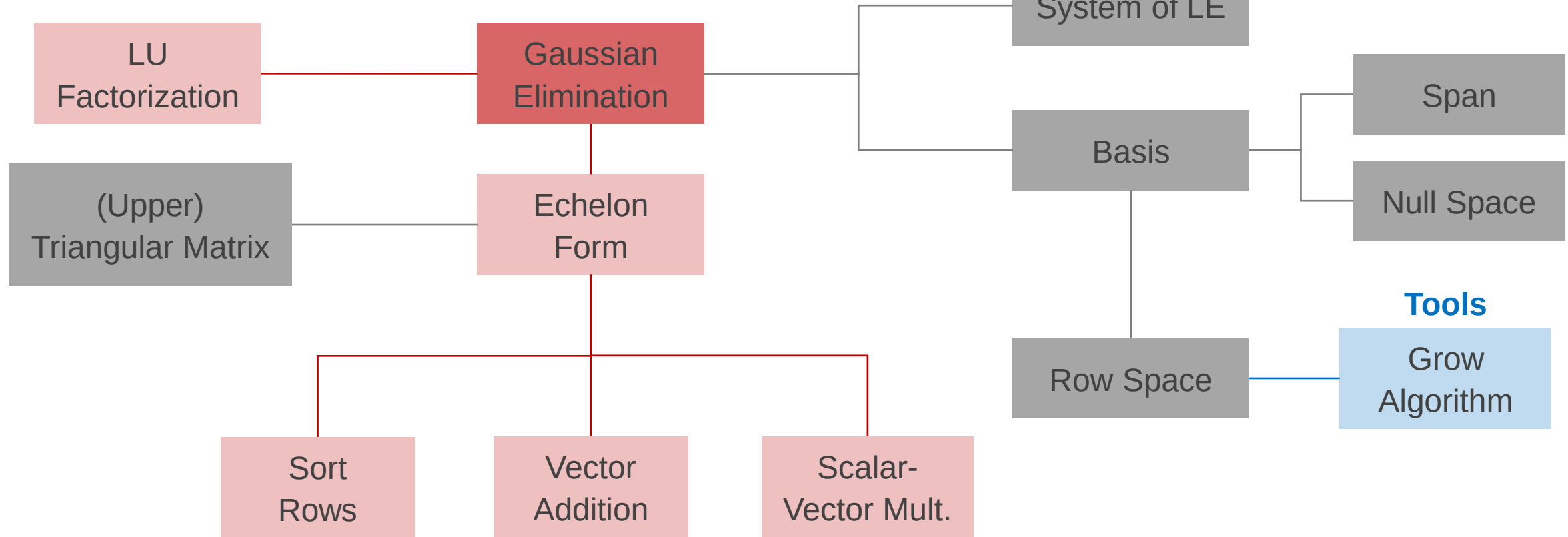
In python using floating-point numbers issues may arise due to **round-off errors** for very small and very large numbers

Solutions are not
covered in detail in
this course



Lecture 15 - Summary

Concepts





Thank You For Your Attention!



**Accomplishment unlocked:
Lecture 15 Completed**



APPENDIX



Failure of Gaussian Elimination (I)

This algorithm is mathematically correct...



Example with very high and very low numbers first by hand

Start

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$$

After Iteration 1

$$\rightarrow C_1 - 10^{20}(C_0) \begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 1 & -1 \end{bmatrix}$$

$$\begin{aligned} &= \begin{bmatrix} 1 & 10^{20} & 1 \end{bmatrix} - 10^{20} \begin{bmatrix} 10^{-20} & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 10^{20} & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 10^{20} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 10^{20} & 1 - 10^{20} \end{bmatrix} \end{aligned}$$

After Iteration 2

$$\rightarrow C_2 - 10^{-20}(C_1) \begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$$

$$\begin{aligned} &= \begin{bmatrix} 0 & 1 & -1 \end{bmatrix} - 10^{-20} \begin{bmatrix} 0 & 10^{20} & 1 - 10^{20} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 1 & -1 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 10^{-20} - 1 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & -10^{-20} \end{bmatrix} \end{aligned}$$



Failure of Gaussian Elimination (II)

This algorithm is mathematically correct...



Example with very high and very low numbers first by hand

Start	After Iteration 1	After Iteration 2	After Iteration 3 [END]
$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$
			(No Change)
New Row-List	New Row-List	New Row-List	
$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$	



Failure of Gaussian Elimination (III)



This algorithm is mathematically correct... But we compute using floating-point numbers!

Start

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$$

After Iteration 1

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 1 & -1 \end{bmatrix}$$

After Iteration 2

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 0 & 0 \end{bmatrix}$$

After Iteration 3 [END]

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 0 & 0 \end{bmatrix}$$

(No Change)

WAIT A MINUTE



New Row-List

$$\begin{bmatrix} 10^{-20} & 0 & 1 \end{bmatrix}$$

New Row-List

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \end{bmatrix}$$

New Row-List

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \end{bmatrix}$$

(No Change)



Failure of Gaussian Elimination (IV)

This algorithm is mathematically correct... But we compute using floating-point numbers!

		After Iteration 1	After Iteration 2	After Iteration 3 [END]
Start $\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$	By hand (Theory)	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$
				(No Change)
		After Iteration 1	After Iteration 2	After Iteration 3 [END]
Start $\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$	Python	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 1 & -1 \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 0 & 0 \end{bmatrix}$
				(No Change)

Gaussian elimination got the wrong answer due to round-off error.



Failure of Gaussian Elimination (V)

Why is this a problem here?

Lemma: If a matrix is in echelon form, the nonzero rows form a basis for the row space.

Start

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 1 & 10^{20} & 1 \\ 0 & 1 & -1 \end{bmatrix}$$

After Iteration 3 [END] 

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & 1 - 10^{20} \\ 0 & 0 & -10^{-20} \end{bmatrix}$$

After Iteration 3 [END] 

$$\begin{bmatrix} 10^{-20} & 0 & 1 \\ 0 & 10^{20} & -10^{20} \\ 0 & 0 & 0 \end{bmatrix}$$

The row space is spanned by three vectors ... but Python suggests two are enough (wrong!)



Failure of Gaussian Elimination (VI)

Gaussian elimination got the wrong answer due to round-off error.

These problems can be mitigated by choosing the pivot element carefully:

- ▶ *Partial pivoting*: Among rows with nonzero entries in column c , choose row with entry having *largest* absolute value.
- ▶ *Complete pivoting*: Instead of selecting order of columns beforehand, in each iteration choose column to maximize absolute value of pivot element.

In this course, we won't study these techniques in detail.